



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN GENERAL DE PREGRADO

PROYECTO DE CARRERA INGENIERÍA EN INFORMÁTICA

CATEDRA: LENGUAJES Y COMPILADORES

Lenguajes y Gramáticas Formales

Profesor:

Félix Márquez

Autores:

Keibel Guilarte

C.I 28.726.602

Oriana Márquez

C.I 31.354.299

Jeanny Monagas

C.I 30.857.471

Victor Vargas

C.I 30.697.219

Ciudad Guayana, junio del 2026.

.1.- Fundamentos y Jerarquía (Teoría Aplicada)

2.2.1.- Relación Gramática-Lenguaje:

Una gramática formal es un conjunto riguroso de reglas matemáticas que definen la estructura de un idioma, mientras que un lenguaje formal es el conjunto de todas las cadenas válidas (palabras) que pueden construirse utilizando exclusivamente esas reglas. El mecanismo mediante el cual una gramática genera un lenguaje comienza siempre desde un símbolo inicial no terminal al cual se le aplican iterativamente las reglas de producción, reemplazando las variables por combinaciones de símbolos terminales y no terminales hasta que la cadena resultante esté compuesta únicamente por caracteres del alfabeto, momento en el que la derivación concluye. Un ejemplo práctico de esta relación es una gramática cuyo símbolo inicial S posea las producciones $S \rightarrow aSb$ y $S \rightarrow ab$, la cual iterativamente generará cadenas que siempre contengan una cantidad idéntica de letras "a" seguidas por la misma cantidad de letras "b", demostrando cómo un modelo matemático dictamina la existencia y los límites de las palabras de un lenguaje.

2.2.2.- Jerarquía de Chomsky:

La Jerarquía de Chomsky clasifica las gramáticas formales en cuatro grandes niveles de acuerdo con las restricciones de sus reglas de producción. El Tipo 0 corresponde a las gramáticas irrestrictas, las cuales no tienen ninguna limitación estructural y pueden ser reconocidas teóricamente por una Máquina de Turing, ilustradas mediante producciones complejas como $\langle A \rangle \langle B \rangle ::= \langle C \rangle \mid a$. El Tipo 1 engloba a las gramáticas sensibles al contexto, donde la longitud del lado izquierdo de la producción siempre debe ser menor o igual a la del lado derecho, lo que significa que el reemplazo depende de los símbolos adyacentes, tal como ocurre en la regla $a \langle B \rangle c ::= a b c$. El Tipo 2 está constituido por las gramáticas libres de contexto, las cuales exigen que su lado izquierdo sea única y exclusivamente un solo símbolo no terminal para permitir la recursión de estructuras anidadas, como se evidencia en la notación $\langle S \rangle ::= a \langle S \rangle b \mid c$. Por último, el Tipo 3 define a las gramáticas regulares, que son las más restrictivas al obligar a que el lado derecho contenga únicamente un símbolo terminal o un terminal seguido de un solo no terminal, facilitando el análisis léxico mediante reglas simples como $\langle S \rangle ::= a \langle B \rangle \mid a$.

2.2.- Derivación y Modelado (Caso Práctico: Genoma)

Explicación del Código de Simulación:

El código en Python implementa una Gramática Libre de Contexto diseñada específicamente para simular el comportamiento de una tortuga gráfica sobre el alfabeto del genoma. El script se organiza estructurando la lógica en componentes secuenciales, comenzando con la definición formal de la gramática a través de un

diccionario que contiene los símbolos no terminales, los terminales y las reglas de producción necesarias para construir las diversas formas geométricas. Seguidamente, el programa incorpora un motor de derivación por la izquierda que procesa una lista de pasos para reemplazar iterativamente la variable más a la izquierda, validando cada regla hasta generar una palabra final compuesta únicamente por caracteres del alfabeto. Una vez obtenida esta cadena, un intérprete traduce cada letra en movimientos bidimensionales, calculando los trazos ortogonales según las instrucciones de avanzar o girar noventa grados. Para concluir el proceso, el código renderiza las coordenadas calculadas mediante una doble salida, mostrando el resultado tanto en una cuadrícula ASCII en la consola como a través de una ventana visual trazada dinámicamente con el módulo turtle.

2.2.1.- Gramática para dibujar (Análisis Teórico y Práctico):

El documento teórico fundamenta el diseño conceptual de la herramienta gráfica estableciendo el alfabeto base del genoma compuesto por los símbolos terminales a, c, g y t, donde cada letra actúa como una orden específica de trazado, rotación o desplazamiento espacial con el lápiz levantado. El texto explica detalladamente cómo funciona el mecanismo de derivación por la izquierda, demostrando que al sustituir progresivamente el símbolo no terminal más a la izquierda de la forma sentencial es posible aprovechar la recursividad de las reglas para generar lados de longitudes arbitrarias y componer estructuras modulares. Toda esta fundamentación analítica se evidencia empíricamente en la imagen adjunta, la cual captura la salida de la interfaz gráfica ilustrando los cinco casos prácticos exigidos que corresponden al cuadrado de lado tres, el rectángulo de cinco por tres, la figura en forma de L, la escalera de tres peldaños y los dos cuadrados separados por un salto. La imagen demuestra que las cadenas generadas por la gramática poseen un significado semántico real, transformando secuencias de texto abstracto en figuras exactas y comprobando de manera directa la relación existente entre el modelo matemático y el lenguaje visual producido.

2.3.- Higiene y Optimización de Gramáticas

2.3.1.- Patologías de las Gramáticas:

Las gramáticas mal diseñadas introducen patologías que causan fallos críticos en los compiladores, siendo el primer caso la ambigüedad, la cual ocurre cuando una misma cadena de entrada puede ser generada mediante dos árboles de derivación estructuralmente distintos y válidos. Por ejemplo, en una gramática de operaciones lógicas con la regla $\langle E \rangle ::= \langle E \rangle + \langle E \rangle \mid \langle E \rangle * \langle E \rangle \mid a$, la cadena $a + a * a$ es patológicamente ambigua porque un árbol agrupa primero la suma evaluando $(a + a) * a$, mientras que el segundo árbol agrupa primero la multiplicación interpretando $a + (a * a)$, dejando al analizador sin una jerarquía de operadores definida.

El segundo caso es la recursividad por la izquierda, un error lógico que surge cuando un símbolo no terminal se llama a sí mismo en la primera posición de su propia regla, provocando que los analizadores descendentes entren en un bucle infinito al intentar procesar la regla $\langle A \rangle ::= \langle A \rangle a \mid b$. El algoritmo para eliminar esta patología consiste en tomar la regla general $A \rightarrow A\alpha \mid \beta$ y transformar la estructura introduciendo un nuevo estado, moviendo el caso base al inicio como $\langle A \rangle ::= b \langle A' \rangle$ y trasladando la parte recursiva hacia la derecha con $\langle A' \rangle ::= a \langle A' \rangle \mid \epsilon$, logrando que el compilador consuma el carácter inicial inmediatamente.

El tercer caso corresponde a la necesidad de factorización por la izquierda, una situación que se presenta cuando múltiples rutas de un mismo no terminal comienzan con exactamente el mismo prefijo, obligando al compilador a tomar decisiones sin suficiente contexto. En el escenario de sentencias condicionales definidas por la regla $\langle S \rangle ::= i \langle E \rangle t \langle S \rangle \mid i \langle E \rangle t \langle S \rangle e \langle S \rangle \mid a$, las dos primeras opciones comparten el gran bloque $i \langle E \rangle t \langle S \rangle$, por lo que la gramática optimizada extrae ese factor común creando la producción principal $\langle S \rangle ::= i \langle E \rangle t \langle S \rangle \langle S' \rangle \mid a$ y aísla la divergencia en el nuevo no terminal $\langle S' \rangle ::= e \langle S \rangle \mid \epsilon$, eliminando así cualquier conflicto de lectura de tokens.

2.4.- De la Expresión al Autómata (Caso Práctico: Ajedrez)

2.4.1.- Notación PGN (Portable Game Notation):

Para abordar el reconocimiento de la notación de ajedrez, el programa implementa un Autómata Finito Determinístico diseñado explícitamente para evaluar cadenas pertenecientes al lenguaje Portable Game Notation. Este autómata se plantea mediante una quintupla formal donde se establece un estado inicial q_0 , un conjunto de estados de aceptación y una función de transición detallada que procesa la entrada símbolo a símbolo. Durante la ejecución del código, la función principal simula la máquina evaluando caracteres clasificados estructuralmente en piezas, columnas, filas o literales, y consulta el diccionario de transiciones en memoria para moverse ininterrumpidamente de un estado a otro. Si el autómata logra consumir toda la cadena y se detiene en un estado de aceptación válido como q_F , q_8 , q_{10} o q_{Chk} , la jugada es reconocida y aceptada exitosamente, mientras que cualquier ausencia de transición para un símbolo detiene el proceso de inmediato rechazando la entrada por ser inválida, demostrando en la práctica el mecanismo exacto que utilizan los analizadores léxicos convencionales dentro de los compiladores modernos.

2.4.2.- Subconjunto simplificado del PGN:

El subconjunto simplificado del lenguaje PGN definido para este modelado abarca de manera exclusiva los movimientos básicos más representativos del ajedrez, agrupando el alfabeto formal en piezas como el rey, la dama, la torre, el alfil y el

caballo, junto con las columnas del tablero de la "a" a la "h" y las filas numeradas del uno al ocho. Dentro de este subconjunto específico se reconocen formalmente las jugadas de pieza convencionales, las jugadas de peón simples mediante la indicación directa de su columna y fila de destino, las capturas de peón que exigen de manera estricta la inclusión del símbolo "x", así como los movimientos especiales correspondientes al enroque corto representado textualmente por "O-O" y el enroque largo denotado como "O-O-O". Además, el modelo contempla estructuralmente la adición de un sufijo opcional al final de cualquier movimiento considerado válido para indicar situaciones de jaque con el símbolo "+" o de jaque mate utilizando el carácter "#", abarcando así un espectro de cadenas prácticas que van desde un simple avance como "e4" hasta combinaciones sintácticas más complejas como "Qh5" o "exd5".

2.4.3.- Expresión Regular y AFD equivalente:

Para consolidar el análisis formal de este lenguaje regular, se diseñó la expresión regular que condensa de forma matemática y compacta todas las variaciones permitidas de jugadas de pieza, avances de peón y enroques con sus respectivos sufijos opcionales. De manera totalmente equivalente, se construyó el Autómata Finito Determinístico correspondiente cuya arquitectura interna se visualiza de forma clara en el diagrama de estados adjunto, el cual muestra cómo desde el estado inicial q_0 el flujo de lectura se bifurca en tres ramas principales e independientes diseñadas para procesar piezas, peones y enroques, convergiendo todas de forma determinista y sin ningún tipo de ambigüedad hacia los estados finales de aceptación. La equivalencia teórica indudable entre ambos modelos matemáticos, los cuales pertenecen inherentemente al Tipo 3 de la Jerarquía de Chomsky, se demuestra empíricamente durante la ejecución del script en Python, donde tanto la expresión regular compilada como el motor de derivación manual del autómata son sometidos a una batería de veinte casos de prueba simultáneos, arrojando exactamente el mismo veredicto de aceptación o rechazo para cada cadena sin presentar absolutamente ninguna discrepancia funcional.